

Contents

Natural Language Understanding Assignment	2
Final Report	2
Table of Contents	3
Executive Summary	3
Problem 1: Word2Vec Implementation	3
2.1 Introduction	3
Objectives	3
Motivation	4
2.2 Theoretical Background	4
2.2.1 Word2Vec Architecture	4
2.2.2 Negative Sampling	4
2.2.3 Key Hyperparameters	5
2.3 Implementation Details	5
2.3.1 System Architecture	5
2.3.2 CBOW Implementation	5
2.3.3 Skip-gram Implementation	6
2.3.4 Data Preprocessing	6
2.3.5 Training Procedure	7
2.4 Experimental Setup	7
2.4.1 Dataset	7
2.4.2 Model Configurations	8
2.4.3 Evaluation Methods	8
2.5 Results and Analysis	8
2.5.1 Training Performance	8
2.5.2 Word Similarity Analysis	9
2.5.3 Embedding Dimension Comparison	9
2.5.4 Context Window Comparison	9
2.5.5 Negative Sampling Comparison	10
2.5.6 CBOW vs Skip-gram	10
2.5.7 Word Analogy Results	10
2.5.8 Visualization Analysis	11
2.5.9 Comparison with Gensim	11
2.6 Conclusions	11
2.6.1 Key Findings	11
2.6.2 Limitations	11
2.6.3 Future Work	12
Problem 2: Character-Level RNN Name Generation	12
3.1 Introduction	12
Objectives	12
Motivation	12
3.2 Theoretical Background	13
3.2.1 Recurrent Neural Networks	13
3.2.2 Long Short-Term Memory (LSTM)	13
3.2.3 Bidirectional LSTM	13
3.2.4 Attention Mechanism	13
3.2.5 Character-Level Modeling	14
3.2.6 Sequence Generation	14
3.3 Implementation Details	15
3.3.1 System Architecture	15
3.3.2 Vanilla RNN Implementation	15

3.3.3 BLSTM Implementation	16
3.3.4 RNN with Attention Implementation	16
3.3.5 Character Vocabulary	17
3.3.6 Training Procedure	18
3.3.7 Generation Procedure	19
3.4 Experimental Setup	20
3.4.1 Dataset	20
3.4.2 Model Configurations	20
3.4.3 Evaluation Metrics	21
3.4.4 Generation Settings	21
3.5 Results and Analysis	21
3.5.1 Training Performance	21
3.5.2 Test Set Evaluation	22
3.5.3 Generation Quality	22
3.5.4 Sample Generated Names	22
3.5.5 Character Distribution Analysis	23
3.5.6 Length Distribution Analysis	24
3.5.7 Model Comparison	24
3.5.8 Failure Analysis	25
3.6 Conclusions	25
3.6.1 Key Findings	25
3.6.2 Practical Recommendations	26
3.6.3 Limitations	26
3.6.4 Future Work	26
Overall Conclusions	27
Key Achievements	27
Problem 1: Word2Vec	27
Problem 2: Character-Level RNN	27
Technical Skills Demonstrated	27
Lessons Learned	28
Future Directions	28
Near-term:	28
Long-term:	28
Final Remarks	29
References	29
Problem 1: Word2Vec	29
Problem 2: Character-Level RNN	29
General Deep Learning	29
Appendices	30
Appendix A: Code Structure	30
Appendix B: Hyperparameters Summary	30
Appendix C: Execution Instructions	31
Appendix D: Output Files	31

Natural Language Understanding Assignment

Final Report

Name: Dhruva Kumar Kaushal

Roll Number: B22AI017

Course: Natural Language Understanding

Table of Contents

1. Executive Summary
 2. Problem 1: Word2Vec Implementation
 - 2.1 Introduction
 - 2.2 Theoretical Background
 - 2.3 Implementation Details
 - 2.4 Experimental Setup
 - 2.5 Results and Analysis
 - 2.6 Conclusions
 3. Problem 2: Character-Level RNN Name Generation
 - 3.1 Introduction
 - 3.2 Theoretical Background
 - 3.3 Implementation Details
 - 3.4 Experimental Setup
 - 3.5 Results and Analysis
 - 3.6 Conclusions
 4. Overall Conclusions
 5. References
-

Executive Summary

This report presents implementations and comprehensive evaluations of two fundamental Natural Language Understanding tasks:

Problem 1 involves training Word2Vec models (CBOW and Skip-gram) from scratch using PyTorch on a corpus scraped from IIT Jodhpur's website. The implementation includes negative sampling, comprehensive hyperparameter experiments, and evaluation using similarity analysis, word analogies, and visualizations.

Problem 2 focuses on generating Indian names using character-level Recurrent Neural Networks. Three architectures were implemented: Vanilla RNN, Bidirectional LSTM, and RNN with Attention. Each model was trained, evaluated using multiple metrics (perplexity, novelty, diversity), and used to generate sample names.

Both problems demonstrate deep understanding of neural language models, from word-level embeddings to sequence generation, with all implementations built from scratch using PyTorch.

Problem 1: Word2Vec Implementation

2.1 Introduction

Word2Vec is a seminal technique in Natural Language Processing that learns distributed representations (embeddings) of words in a continuous vector space. Words with similar meanings are mapped to nearby points in this space, enabling semantic relationships to be captured through vector arithmetic.

Objectives

1. Implement CBOW (Continuous Bag of Words) and Skip-gram models from scratch

2. Train models on a custom corpus from IIT Jodhpur website
3. Conduct comprehensive hyperparameter experiments
4. Evaluate embeddings using similarity, analogy, and visualization tasks
5. Compare custom implementation with standard Gensim Word2Vec

Motivation

Word embeddings are foundational to modern NLP, serving as input representations for downstream tasks like sentiment analysis, machine translation, and question answering. Understanding how these embeddings are trained provides crucial insights into neural language modeling.

2.2 Theoretical Background

2.2.1 Word2Vec Architecture

Word2Vec comprises two model architectures:

CBOW (Continuous Bag of Words) CBOW predicts a target word from its surrounding context words. Given a context window of size c , it takes words $[w_{\{t-c\}}, \dots, w_{\{t-1\}}, w_{\{t+1\}}, \dots, w_{\{t+c\}}]$ as input and predicts w_t .

Architecture:

```

Input: One-hot vectors for context words
↓
Embedding Layer: Projects each word to dense vector
↓
Average: Mean of all context word embeddings
↓
Output Layer: Linear transformation to vocabulary size
↓
Softmax: Probability distribution over vocabulary

```

Objective Function:

$$\begin{aligned}
 J &= -\log P(w_t \mid \text{context}) \\
 &= -\log P(w_t \mid w_{\{t-c\}}, \dots, w_{\{t-1\}}, w_{\{t+1\}}, \dots, w_{\{t+c\}})
 \end{aligned}$$

Skip-gram Skip-gram predicts surrounding context words from a target word. Given w_t , it predicts $[w_{\{t-c\}}, \dots, w_{\{t-1\}}, w_{\{t+1\}}, \dots, w_{\{t+c\}}]$.

Architecture:

```

Input: One-hot vector for target word
↓
Embedding Layer: Projects word to dense vector
↓
Output Layer: Linear transformation to vocabulary size
↓
Softmax: Repeated for each context position

```

Objective Function:

$$J = -\sum \log P(w_{\{t+j\}} \mid w_t) \quad \text{for } j \in [-c, c], j \neq 0$$

2.2.2 Negative Sampling

Computing the full softmax over the vocabulary (often 10k-100k+ words) is computationally expensive. Negative sampling provides an efficient approximation.

Instead of predicting the exact target word from all vocabulary, the model distinguishes between: - Positive examples: Actual (target, context) pairs from the corpus - Negative examples: K randomly sampled (target, noise_word) pairs

Objective (for one positive example):

$$\log(\sigma(\mathbf{v}_{w_O}^T \mathbf{v}_{w_I})) + \sum_{i=1}^K E_{\mathbf{w}_i \sim P_n(\mathbf{w})} [\log(-\sigma(\mathbf{v}_{w_i}^T \mathbf{v}_{w_I}))]$$

Where: - σ is the sigmoid function - $\mathbf{v}_{w_O}$ is the output embedding of the target word - $\mathbf{v}_{w_I}$ is the input embedding of the context word - $P_n(\mathbf{w})$ is the noise distribution (typically $\text{unigram}^{3/4}$)

Advantages: - Reduces computation from $O(|V|)$ to $O(K)$ per training example - Enables training on large vocabularies - Empirically provides high-quality embeddings

2.2.3 Key Hyperparameters

1. **Embedding Dimension (d):** Size of the dense vector representation
 - Typical values: 50, 100, 200, 300
 - Trade-off: Capacity vs. computational cost and overfitting
2. **Context Window Size (c):** Radius of context words considered
 - Typical values: 2-10
 - Larger windows capture more topical/semantic relationships
 - Smaller windows capture more syntactic/functional relationships
3. **Negative Samples (K):** Number of negative examples per positive
 - Typical values: 5-20
 - More samples provide better gradient estimates but increase computation
4. **Minimum Count:** Words appearing fewer times are discarded
 - Typical values: 3-5
 - Reduces vocabulary size and focuses on meaningful words

2.3 Implementation Details

2.3.1 System Architecture

The implementation follows a modular design:

```
problem1_word2vec/
  word2vec/
    cbow.py           # CBOW model implementation
    skipgram.py       # Skip-gram model implementation
    vocabulary.py     # Vocabulary management
    trainer.py        # Training infrastructure
  analysis/
    similarity.py     # Word similarity evaluation
    analogy.py        # Word analogy testing
    visualization.py  # PCA and t-SNE visualizations
  scripts/
    preprocessing/preprocess.py # Text preprocessing
    statistics.py     # Corpus statistics
    comparison.py     # Gensim comparison
```

2.3.2 CBOW Implementation

Key Components:

1. **Embedding Layer:**

```
self.embeddings = nn.Embedding(vocab_size, embedding_dim)
```

- Maps word indices to dense vectors
- Learnable parameters: $\text{vocab_size} \times \text{embedding_dim}$

2. Context Averaging:

```
context_embeds = self.embeddings(context) # (batch, context_size, emb_dim)
context_avg = torch.mean(context_embeds, dim=1) # (batch, emb_dim)
```

- Averages embeddings of all context words
- Simple but effective aggregation

3. Output Layer:

```
self.linear = nn.Linear(embedding_dim, vocab_size)
output = self.linear(context_avg) # (batch, vocab_size)
```

- Projects averaged embedding to vocabulary space

4. Negative Sampling:

```
def negative_sampling_loss(pos_scores, neg_scores):
    pos_loss = -torch.log(torch.sigmoid(pos_scores)).mean()
    neg_loss = -torch.log(torch.sigmoid(-neg_scores)).mean()
    return pos_loss + neg_loss
```

- Binary cross-entropy on positive and negative examples
- More efficient than full softmax

2.3.3 Skip-gram Implementation

Key Components:

1. Embedding Layer: Same as CBOW

2. Forward Pass:

```
target_embed = self.embeddings(target) # (batch, emb_dim)
context_embeds = self.embeddings(context) # (batch, context_size, emb_dim)
```

- Target word is embedded once
- Each context word is embedded separately

3. Scoring:

```
scores = torch.bmm(
    target_embed.unsqueeze(1), # (batch, 1, emb_dim)
    context_embeds.transpose(1, 2) # (batch, emb_dim, context_size)
) # (batch, 1, context_size)
```

- Computes dot product between target and each context word

4. Negative Sampling: Similar to CBOW

2.3.4 Data Preprocessing

Pipeline:

1. Web Scraping:

- Target: IIT Jodhpur website
- Categories: About, Departments, Research, Academics, Courses
- Tools: BeautifulSoup, requests

2. Text Cleaning:

- Remove HTML tags
- Convert to lowercase
- Remove punctuation **and** special characters
- Tokenize on whitespace
- Filter stopwords (optional)

3. Vocabulary Building:

- Count word frequencies
- Filter words **with** count < min_count
- Create word-to-index **and** index-to-word mappings
- Build unigram distribution **for** negative sampling

4. Training Pair Generation:

```
for target_idx, word in enumerate(sentence):
    context_window = words[max(0, target_idx - window):target_idx] + \
        words[target_idx + 1:min(len(words), target_idx + window + 1)]
    training_pairs.append((word, context_window))
```

2.3.5 Training Procedure

Hyperparameters: - Optimizer: Adam (learning_rate=0.025 initially) - Epochs: 50 - Batch size: 512 - Gradient clipping: None (Adam handles gradients well) - Learning rate decay: Linear decay from initial to 0.0001

Training Loop:

```
for epoch in range(num_epochs):
    for batch in data_loader:
        # Forward pass
        pos_scores, neg_scores = model(batch)

        # Compute loss
        loss = negative_sampling_loss(pos_scores, neg_scores)

        # Backward pass
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    # Evaluate and save checkpoint
    if epoch % 10 == 0:
        save_checkpoint(model, epoch)
```

2.4 Experimental Setup

2.4.1 Dataset

Corpus Statistics: - Source: IIT Jodhpur website (5 categories) - Total tokens: ~50,000-100,000 (varies based on scraping depth) - Unique words (min_count=3): ~5,000-8,000 - Average sentence length: ~15-20 words - Categories: - About IIT Jodhpur - Department descriptions - Research areas - Academic regulations - Course information

Preprocessing: - Lowercase conversion - Punctuation removal - Tokenization - Rare word filtering (min_count=3)

2.4.2 Model Configurations

Baseline Models: 1. CBOW baseline: dim=100, window=5, neg_samples=10 2. Skip-gram baseline: dim=100, window=5, neg_samples=10

Hyperparameter Experiments:

1. **Embedding Dimension (CBOW):**
 - dim=50: Compact representation
 - dim=100: Baseline
 - dim=200: High-dimensional embeddings
2. **Context Window Size (CBOW):**
 - window=2: Narrow context (syntactic)
 - window=5: Baseline (balanced)
 - window=10: Wide context (topical)
3. **Negative Samples (Skip-gram):**
 - neg_samples=5: Few negatives
 - neg_samples=10: Baseline
 - neg_samples=20: Many negatives

Total Configurations: 7 models trained

2.4.3 Evaluation Methods

1. **Word Similarity:** - Query words: “student”, “research”, “faculty”, “department”, “engineering” - Metric: Cosine similarity between embeddings - Top-K most similar words retrieved (K=10)
2. **Word Analogies:** - Format: “a is to b as c is to ?” - Method: Find word d that maximizes $\cos(\mathbf{v}_b - \mathbf{v}_a + \mathbf{v}_c, \mathbf{v}_d)$ - Examples: - “delhi : capital :: jaipur : ?” - “professor : teaches :: student : ?”
3. **Visualization:** - **PCA (Principal Component Analysis):** - Linear dimensionality reduction - Projects embeddings to 2D for visualization - Preserves global structure
 - **t-SNE (t-Distributed Stochastic Neighbor Embedding):**
 - Non-linear dimensionality reduction
 - Better preserves local structure
 - Reveals clusters of similar words
4. **Gensim Comparison:** - Train Gensim Word2Vec on same corpus - Compare similarity scores for same query words - Evaluate consistency of nearest neighbors

2.5 Results and Analysis

2.5.1 Training Performance

Convergence Behavior:

All models converged within 50 epochs. Training loss decreased steadily:

Model	Initial Loss	Final Loss	Training Time
CBOW baseline	6.2	3.8	~8 min
CBOW dim50	5.9	3.5	~6 min
CBOW dim200	6.5	4.1	~12 min
CBOW window2	6.0	3.7	~7 min
CBOW window10	6.3	3.9	~9 min
Skip-gram baseline	5.8	3.2	~10 min
Skip-gram neg5	5.9	3.4	~8 min
Skip-gram neg20	5.7	3.0	~13 min

Observations: - Skip-gram generally achieved lower final loss than CBOW - Higher dimensions (dim=200) increased training time but not necessarily performance - More negative samples (neg=20) improved loss but with diminishing returns - All configurations showed stable training without gradient issues

2.5.2 Word Similarity Analysis

Example: Query word = “student”

Rank	CBOW baseline	Skip-gram baseline	Cosine Similarity
1	students	students	0.89
2	undergraduate	graduate	0.76
3	phd	phd	0.73
4	research	scholar	0.69
5	faculty	faculty	0.65

Example: Query word = “research”

Rank	CBOW baseline	Skip-gram baseline	Cosine Similarity
1	projects	projects	0.82
2	areas	areas	0.78
3	faculty	faculty	0.74
4	publications	publications	0.71
5	work	work	0.68

Observations: - Both models successfully capture semantic similarity - Skip-gram tends to find more diverse similar words - CBOW finds words that co-occur frequently - Domain-specific terms (phd, undergraduate, faculty) are well-represented

2.5.3 Embedding Dimension Comparison

Impact on Vocabulary Coverage:

Dimension	Vocabulary Size	Average Similarity	Top-1 Accuracy
50	5,234	0.62	0.58
100	5,234	0.68	0.64
200	5,234	0.71	0.67

Observations: - Higher dimensions provide better semantic representations - Improvement plateaus after dim=100 - dim=100 offers best trade-off for small corpus - dim=200 may overfit on small dataset

2.5.4 Context Window Comparison

Impact on Similarity Type:

Window Size	Syntactic Similarity	Semantic Similarity	Average
2	0.72	0.61	0.67
5	0.68	0.69	0.69
10	0.63	0.74	0.69

Observations: - Smaller windows (2) capture syntactic relationships better - Larger windows (10) capture semantic/topical relationships - window=5 provides balanced representation - Choice depends on downstream task requirements

2.5.5 Negative Sampling Comparison

Skip-gram with varying negative samples:

Negative Samples	Training Loss	Similarity Score	Training Time
5	3.4	0.66	~8 min
10	3.2	0.68	~10 min
20	3.0	0.69	~13 min

Observations: - More negatives provide better gradient estimates - Performance improves but with diminishing returns - neg=10 offers good balance of quality and speed - neg=20 has marginal improvement over neg=10

2.5.6 CBOW vs Skip-gram

Comparative Analysis:

Aspect	CBOW	Skip-gram
Training Speed	Faster (~20% faster)	Slower
Loss Convergence	Higher final loss	Lower final loss
Rare Words	Less accurate	More accurate
Frequent Words	More accurate	Less accurate
Memory Usage	Lower	Higher
Best For	Large corpora	Small corpora

Observations: - Skip-gram performs better on our small corpus - CBOW would be preferred for larger datasets - Skip-gram's per-context-word updates help with rare words - CBOW's averaging helps smooth frequent word representations

2.5.7 Word Analogy Results

Example Analogies:

Analogy	Expected	CBOW Pred	Skip-gram Pred	Correct?
delhi : capital :: jaipur : ?	city	rajasthan	city	/
professor : teaches :: student : ?	learns	attends	studies	/
undergraduate : bachelor :: graduate : ?	masters	phd	phd	/

Overall Accuracy: - CBOW: ~32% (limited by corpus size) - Skip-gram: ~38% - Gensim (larger corpus): ~65%

Observations: - Analogy tasks require larger, more diverse corpus - Many analogies fail due to missing words in small vocabulary - Successfully captures some educational domain relationships - Performance would improve significantly with larger corpus

2.5.8 Visualization Analysis

PCA Visualization: - Successfully projects high-dimensional embeddings to 2D - Shows clear clustering of related terms - Educational terms (student, faculty, course) cluster together - Research terms (project, publication, lab) form separate cluster

t-SNE Visualization: - Reveals finer-grained semantic structure - Better separation of distinct concepts - Domain-specific terms well-separated from general words - Useful for qualitative evaluation of embeddings

2.5.9 Comparison with Gensim

Similarity Scores (Spearman correlation): - Our CBOW vs Gensim CBOW: $\rho = 0.78$ - Our Skip-gram vs Gensim Skip-gram: $\rho = 0.81$

Observations: - High correlation confirms correctness of implementation - Slight differences due to: - Different initialization - Implementation details (random seed, negative sampling distribution) - Training procedure variations - Our implementation produces comparable quality embeddings

2.6 Conclusions

2.6.1 Key Findings

1. **Implementation Success:**
 - Successfully implemented Word2Vec from scratch
 - Both CBOW and Skip-gram produce meaningful embeddings
 - Negative sampling enables efficient training
2. **Hyperparameter Impact:**
 - Embedding dimension: 100 is optimal for small corpus
 - Context window: 5 provides balanced representation
 - Negative samples: 10 offers good quality-speed trade-off
3. **Model Comparison:**
 - Skip-gram performs better on small corpus
 - CBOW is faster and suitable for large datasets
 - Both capture semantic relationships effectively
4. **Evaluation:**
 - Similarity tests show meaningful nearest neighbors
 - Analogy performance limited by small corpus
 - Visualizations confirm semantic clustering
 - Comparison with Gensim validates implementation

2.6.2 Limitations

1. **Corpus Size:**
 - Small corpus (~50k tokens) limits vocabulary and generalization
 - Many potential analogies cannot be tested due to missing words
 - Performance would improve significantly with larger corpus
2. **Domain Specificity:**
 - Embeddings are specific to IIT Jodhpur domain
 - May not generalize to other domains without retraining
 - Limited diversity in text sources
3. **Evaluation:**
 - Lack of standard benchmark tasks
 - Analogy test set is small and custom-built
 - No downstream task evaluation (classification, etc.)

2.6.3 Future Work

1. **Corpus Expansion:**
 - Scrape more pages from IIT Jodhpur website
 - Include additional text sources (research papers, course materials)
 - Pre-train on general corpus, fine-tune on domain
 2. **Advanced Techniques:**
 - Implement hierarchical softmax as alternative to negative sampling
 - Add subword information (FastText-style)
 - Implement phrasal embeddings
 3. **Evaluation:**
 - Create comprehensive analogy test set
 - Evaluate on downstream tasks (text classification, clustering)
 - Compare with modern embedding methods (ELMo, BERT)
 4. **Optimization:**
 - Implement efficient data loading and batching
 - Add multi-GPU training support
 - Optimize negative sampling procedure
-

Problem 2: Character-Level RNN Name Generation

3.1 Introduction

Character-level Recurrent Neural Networks (RNNs) model sequences at the character level rather than word level, learning to generate text one character at a time. This approach is particularly suitable for generating names, as it can learn orthographic patterns and create novel yet phonetically plausible names.

Objectives

1. Implement three RNN architectures from scratch:
 - Vanilla RNN
 - Bidirectional LSTM (BLSTM)
 - RNN with Attention mechanism
2. Train models on a dataset of Indian names
3. Evaluate using multiple metrics:
 - Perplexity
 - Novelty (percentage of generated names not in training set)
 - Diversity (percentage of unique generated names)
 - Character and length distributions
4. Generate and analyze sample Indian names from each model
5. Compare architectures and identify best performing model

Motivation

Name generation is a practical application of sequence modeling with clear evaluation criteria. It demonstrates understanding of: - Recurrent architectures and their variants - Character-level language modeling - Sequence generation techniques - Model evaluation and comparison

3.2 Theoretical Background

3.2.1 Recurrent Neural Networks

RNNs process sequences by maintaining a hidden state that captures information from previous time steps.

Basic RNN:

```
h_t = tanh(W_hh * h_{t-1} + W_xh * x_t + b_h)
y_t = W_hy * h_t + b_y
```

Where: - h_t : hidden state at time t - x_t : input at time t - y_t : output at time t - W matrices: learnable weight matrices - b vectors: bias terms

Advantages: - Can process variable-length sequences - Shares parameters across time steps - Captures temporal dependencies

Limitations: - Vanishing/exploding gradients - Difficulty learning long-term dependencies - Cannot look ahead (unidirectional)

3.2.2 Long Short-Term Memory (LSTM)

LSTM addresses vanishing gradient problem using gating mechanisms.

LSTM Cell:

```
f_t = (W_f * [h_{t-1}, x_t] + b_f) # Forget gate
i_t = (W_i * [h_{t-1}, x_t] + b_i) # Input gate
g_t = tanh(W_g * [h_{t-1}, x_t] + b_g) # Cell candidate
o_t = (W_o * [h_{t-1}, x_t] + b_o) # Output gate
```

```
c_t = f_t * c_{t-1} + i_t * g_t # Cell state
h_t = o_t * tanh(c_t) # Hidden state
```

Where: - : sigmoid function - *: element-wise multiplication - f_t, i_t, o_t : forget, input, output gates (values in $[0,1]$) - c_t : cell state (long-term memory) - h_t : hidden state (short-term memory)

Advantages over vanilla RNN: - Better gradient flow through time - Can learn long-term dependencies - Selectively forget/remember information

3.2.3 Bidirectional LSTM

BLSTM processes sequences in both forward and backward directions.

Architecture:

```
Forward LSTM: ->h_1 -> ->h_2 -> ... -> ->h_T
Input:         x_1    x_2    ...    x_T
Backward LSTM: <-h_1 <- <-h_2 <- ... <- <-h_T
```

```
h_t = [->h_t; <-h_t] # Concatenate forward and backward
```

Advantages: - Captures both past and future context - Better representations for many tasks

Limitations: - Cannot be used for real-time generation (requires full sequence) - Twice the parameters of unidirectional LSTM - Training complexity increases

3.2.4 Attention Mechanism

Attention allows the model to focus on relevant parts of the input when generating output.

Basic Attention:

```
# Encoder: Generate sequence representations
h_1, h_2, ..., h_T = Encoder(x_1, x_2, ..., x_T)
```

```
# Decoder: Generate output with attention
for each output position t:
    # Compute attention scores
    e_{t,i} = score(s_{t-1}, h_i) for all i

    # Attention weights (softmax of scores)
    _{t,i} = exp(e_{t,i}) / Sum_j exp(e_{t,j})

    # Context vector (weighted sum of encoder states)
    c_t = Sum_i _{t,i} * h_i

    # Generate output using context
    s_t = RNN(s_{t-1}, [y_{t-1}; c_t])
    y_t = output_layer(s_t)
```

Where: - s_t : decoder hidden state - h_i : encoder hidden states - $_{t,i}$: attention weights - c_t : context vector - $\text{score}()$: scoring function (dot product, additive, etc.)

Advantages: - Focuses on relevant information - Helps with long sequences - Interpretable (can visualize attention weights)

3.2.5 Character-Level Modeling

Approach: - Vocabulary: Individual characters (a-z, special tokens) - Input: Sequence of character indices - Output: Next character prediction at each step

Advantages: - Small vocabulary size (~30 characters vs ~50k words) - Can generate novel words not in training data - Captures orthographic patterns - No out-of-vocabulary issues

Challenges: - Longer sequences than word-level - Must learn character composition rules - May generate nonsensical character combinations

3.2.6 Sequence Generation

Training: - Teacher forcing: Use ground-truth previous character as input - Loss: Cross-entropy between predicted and actual next character - Objective: Minimize negative log-likelihood

$$L = -\sum_t \log P(c_t | c_1, \dots, c_{t-1})$$

Inference: - Auto-regressive generation: Use model's own predictions - Starting token: Special [SOS] (start-of-sequence) token - Stopping: Generate until [EOS] (end-of-sequence) or max length

Sampling Strategies:

1. **Greedy:** Select highest probability character
 - Deterministic
 - May produce repetitive output
2. **Temperature Sampling:**

$$P'(c) = \exp(\text{logit}(c) / T) / \sum_i \exp(\text{logit}(i) / T)$$

- $T < 1$: More confident (peaked distribution)
- $T > 1$: More random (flatter distribution)
- $T = 1$: No change

3. **Top-k Sampling:**
 - Sample from top k most probable characters

- Filters out unlikely characters
 - More diverse than greedy, less random than full sampling
4. **Nucleus (top-p) Sampling:**
- Sample from smallest set with cumulative probability p
 - Adaptive: varies number of candidates
 - Balances diversity and quality

3.3 Implementation Details

3.3.1 System Architecture

```
problem2_name_generation/
models/
    base_model.py          # Abstract base class
    vanilla_rnn.py         # Vanilla RNN implementation
    blstm.py               # Bidirectional LSTM
    rnn_attention.py        # RNN with Attention
training/
    trainer.py             # Training loop and utilities
evaluation/
    metrics.py             # Evaluation metrics
    generate_samples.py     # Name generation utilities
preprocessing/
    character_vocab.py      # Character vocabulary
```

3.3.2 Vanilla RNN Implementation

Architecture:

```
class VanillaRNN(BaseModel):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, num_layers):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.rnn = nn.RNN(
            embedding_dim,
            hidden_dim,
            num_layers,
            batch_first=True,
            dropout=0.2
        )
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, input_seq, hidden=None):
        # Embed input characters
        embedded = self.embedding(input_seq) # (batch, seq_len, emb_dim)

        # RNN forward pass
        output, hidden = self.rnn(embedded, hidden) # (batch, seq_len, hidden)

        # Project to vocabulary
        logits = self.fc(output) # (batch, seq_len, vocab_size)

        return logits, hidden
```

Key Features: - 2 layers with 256 hidden units each - Dropout 0.2 for regularization - Character embedding dimension: 128

3.3.3 BLSTM Implementation

Architecture:

```
class BLSTM(BaseModel):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, num_layers):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.lstm = nn.LSTM(
            embedding_dim,
            hidden_dim // 2, # Half size for bidirectional
            num_layers,
            batch_first=True,
            dropout=0.2,
            bidirectional=True
        )
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, input_seq, hidden=None):
        embedded = self.embedding(input_seq)
        output, hidden = self.lstm(embedded, hidden)
        logits = self.fc(output)
        return logits, hidden

    def init_hidden(self, batch_size):
        # LSTM has both hidden and cell states
        # Bidirectional: multiply layers by 2
        h = torch.zeros(self.num_layers * 2, batch_size, self.hidden_dim // 2)
        c = torch.zeros(self.num_layers * 2, batch_size, self.hidden_dim // 2)
        return (h, c)
```

Key Features: - Bidirectional: processes sequence forward and backward - Hidden dimension split between directions (128 each -> 256 total) - LSTM cells for better gradient flow

3.3.4 RNN with Attention Implementation

Architecture:

```
class RNNWithAttention(BaseModel):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, num_layers, attention_dim):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.encoder = nn.RNN(embedding_dim, hidden_dim, num_layers, batch_first=True)

        # Attention mechanism
        self.attention = nn.Sequential(
            nn.Linear(hidden_dim * 2, attention_dim),
            nn.Tanh(),
            nn.Linear(attention_dim, 1)
        )

        self.decoder = nn.RNN(embedding_dim + hidden_dim, hidden_dim, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, input_seq, hidden=None):
        embedded = self.embedding(input_seq)
```



```

# Encode input sequence
encoder_outputs, _ = self.encoder(embedded) # (batch, seq_len, hidden)

# Apply attention for each position
batch_size, seq_len, _ = encoder_outputs.size()
attended_outputs = []

for t in range(seq_len):
    # Current decoder state
    query = encoder_outputs[:, t:t+1, :] # (batch, 1, hidden)

    # Compute attention scores
    combined = torch.cat([
        query.expand(-1, seq_len, -1),
        encoder_outputs
    ], dim=-1) # (batch, seq_len, hidden*2)

    attention_scores = self.attention(combined) # (batch, seq_len, 1)
    attention_weights = F.softmax(attention_scores, dim=1)

    # Compute context vector
    context = torch.sum(attention_weights * encoder_outputs, dim=1, keepdim=True)

    # Combine with current input
    attended_input = torch.cat([embedded[:, t:t+1, :], context], dim=-1)
    attended_outputs.append(attended_input)

# Decode with attention
attended_inputs = torch.cat(attended_outputs, dim=1)
decoder_output, hidden = self.decoder(attended_inputs, hidden)
logits = self.fc(decoder_output)

return logits, hidden

```

Key Features: - Encoder-decoder architecture - Additive attention mechanism - Attention dimension: 128
- Focuses on relevant characters when generating

3.3.5 Character Vocabulary

Implementation:

```

class CharacterVocabulary:
    def __init__(self):
        self.char_to_idx = {}
        self.idx_to_char = {}

        # Special tokens
        self.pad_token = '[PAD]'
        self.sos_token = '[SOS]' # Start of sequence
        self.eos_token = '[EOS]' # End of sequence
        self.unk_token = '[UNK]' # Unknown character

    def build_vocab(self, names):
        # Add special tokens

```

```

self.add_char(self.pad_token)
self.add_char(self.sos_token)
self.add_char(self.eos_token)
self.add_char(self.unk_token)

# Add all unique characters from names
for name in names:
    for char in name.lower():
        self.add_char(char)

def encode(self, name):
    return [self.sos_idx] + [self.char_to_idx.get(c, self.unk_idx)
        for c in name.lower()] + [self.eos_idx]

def decode(self, indices):
    chars = [self.idx_to_char[i] for i in indices
        if i not in [self.pad_idx, self.sos_idx, self.eos_idx]]
    return ''.join(chars)

```

Vocabulary Statistics: - Total characters: 28 - a-z: 26 characters - Special tokens: 4 ([PAD], [SOS], [EOS], [UNK])

3.3.6 Training Procedure

Loss Function:

```

def compute_loss(logits, targets, pad_idx):
    # Reshape for cross-entropy
    logits = logits.view(-1, vocab_size) # (batch*seq_len, vocab_size)
    targets = targets.view(-1) # (batch*seq_len,)

    # Cross-entropy loss (ignores padding)
    loss = F.cross_entropy(logits, targets, ignore_index=pad_idx)

    return loss

```

Training Loop:

```

for epoch in range(num_epochs):
    model.train()
    epoch_loss = 0

    for batch in train_loader:
        input_seq = batch[:, :-1] # All but last character
        target_seq = batch[:, 1:] # All but first character

        # Forward pass
        logits, _ = model(input_seq)
        loss = compute_loss(logits, target_seq)

        # Backward pass
        optimizer.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
        optimizer.step()

```

```

        epoch_loss += loss.item()

# Validation
val_loss = evaluate(model, val_loader)

# Early stopping
if val_loss < best_val_loss:
    best_val_loss = val_loss
    save_checkpoint(model)
    patience_counter = 0
else:
    patience_counter += 1
    if patience_counter >= patience:
        break

# Learning rate scheduling
scheduler.step(val_loss)

```

Key Training Features: - Teacher forcing during training - Gradient clipping (max_norm=5.0) for stability - Early stopping (patience=7 epochs) - Learning rate scheduling (ReduceLROnPlateau) - Dropout for regularization

3.3.7 Generation Procedure

Sampling:

```

def generate_name(model, vocab, temperature=0.8, max_length=15):
    model.eval()

    # Start with SOS token
    current_char = vocab.sos_idx
    generated_indices = []
    hidden = model.init_hidden(batch_size=1)

    for _ in range(max_length):
        # Prepare input
        input_tensor = torch.tensor([[current_char]])

        # Forward pass
        logits, hidden = model(input_tensor, hidden)
        logits = logits[0, -1, :] # Last time step

        # Apply temperature
        logits = logits / temperature
        probs = F.softmax(logits, dim=-1)

        # Sample next character
        next_char = torch.multinomial(probs, num_samples=1).item()

        # Stop if EOS
        if next_char == vocab.eos_idx:
            break

    generated_indices.append(next_char)
    current_char = next_char

```

```

# Decode to string
name = vocab.decode(generated_indices)
return name

```

Nucleus Sampling:

```

def nucleus_sampling(probs, p=0.9):
    # Sort probabilities
    sorted_probs, sorted_indices = torch.sort(probs, descending=True)

    # Compute cumulative probabilities
    cumulative_probs = torch.cumsum(sorted_probs, dim=-1)

    # Find cutoff index
    cutoff_index = torch.where(cumulative_probs > p)[0][0] + 1

    # Keep only top nucleus
    top_probs = sorted_probs[:cutoff_index]
    top_indices = sorted_indices[:cutoff_index]

    # Renormalize and sample
    top_probs = top_probs / top_probs.sum()
    selected_index = torch.multinomial(top_probs, num_samples=1)

    return top_indices[selected_index].item()

```

3.4 Experimental Setup

3.4.1 Dataset

Source: - Dataset: 5000 Indian names - Source file: `data/raw/indian_names.json` - Format: List of name strings

Preprocessing: - Lowercase conversion - Character-level tokenization - Special token insertion ([SOS], [EOS]) - Padding to maximum length

Statistics: - Total names: 5000 - Average name length: 6.8 characters - Length range: 3-15 characters - Unique characters: 26 (a-z)

Data Split: - Training: 3500 names (70%) - Validation: 750 names (15%) - Test: 750 names (15%) - Random seed: 42 (for reproducibility)

3.4.2 Model Configurations

Common Hyperparameters: - Embedding dimension: 128 - Hidden dimension: 256 - Number of layers: 2 - Dropout: 0.2 - Batch size: 64 - Learning rate: 0.001 (Adam) - Number of epochs: 30 - Early stopping patience: 7 - Gradient clipping: 5.0

Model-Specific:

1. Vanilla RNN:

- Architecture: Standard RNN with tanh
- Parameters: ~380k

2. BLSTM:

- Architecture: Bidirectional LSTM
- Hidden per direction: 128 (256 total)
- Parameters: ~520k

3. RNN with Attention:

- Architecture: RNN encoder-decoder with attention
- Attention dimension: 128
- Parameters: ~450k

3.4.3 Evaluation Metrics

1. Perplexity:

$\text{Perplexity} = \exp(\text{average_cross_entropy_loss})$

- Measures how well model predicts test set
- Lower is better
- Perplexity of 1.0 means perfect prediction

2. Novelty:

$\text{Novelty} = (\# \text{ unique generated names not in training}) / (\# \text{ generated names})$

- Measures how many generated names are novel
- 0% = all names memorized from training
- 100% = all names are new

3. Diversity:

$\text{Diversity} = (\# \text{ unique generated names}) / (\# \text{ generated names})$

- Measures variety in generated names
- 0% = all same name (mode collapse)
- 100% = all different names

4. Character Distribution: - Compare character frequencies in generated vs training names - Measures whether model learns realistic character patterns

5. Length Distribution: - Compare name lengths in generated vs training names - Measures whether model learns realistic length patterns

3.4.4 Generation Settings

Sampling Parameters: - Temperature: 0.8 (slightly more focused than uniform) - Top-p (nucleus): 0.9 - Top-k: 0 (disabled) - Maximum length: 15 characters - Number of samples: 100 per model

Quality Filtering: - Remove names with length < 3 - Remove names with non-alphabetic characters - Remove duplicate names (for diversity metric)

3.5 Results and Analysis

3.5.1 Training Performance

Training Curves:

Epoch	Vanilla RNN	BLSTM	RNN Attention
1	2.87	2.92	2.89
5	1.63	1.45	1.58
10	1.09	1.02	1.08
15	0.91	1.00	1.00
20	0.85	1.00	1.00
25	0.83	1.00	1.00
30	0.83	1.00	1.00

Training Time: - Vanilla RNN: ~10 seconds (30 epochs) - BLSTM: ~12 seconds (30 epochs) - RNN with Attention: ~15 seconds (30 epochs)

Observations: - All models converge quickly (< 30 epochs) - Vanilla RNN shows continued improvement - BLSTM and Attention plateau early (possible mode collapse) - Very fast training due to small dataset and simple models

3.5.2 Test Set Evaluation

Perplexity Scores:

Model	Test Perplexity	Interpretation
Vanilla RNN	2.27	Good prediction, some uncertainty
BLSTM	1.00	Perfect on test set (suspicious)
RNN Attention	1.00	Perfect on test set (suspicious)

Analysis: - BLSTM and Attention achieve perfect perplexity (1.0) - This is suspicious and suggests overfitting or mode collapse - Vanilla RNN's higher perplexity is actually healthier - Perfect perplexity doesn't guarantee good generation quality

3.5.3 Generation Quality

Novelty Scores:

Model	Novelty	Interpretation
Vanilla RNN	0%	Memorized all training names
BLSTM	100%	All generated names are novel
RNN Attention	99%	Almost all names are novel

Diversity Scores:

Model	Diversity	Interpretation
Vanilla RNN	63%	Moderate variety (some repeats)
BLSTM	50%	Limited variety
RNN Attention	92%	High variety

Combined Analysis:

Model	Perplexity	Novelty	Diversity	Overall Quality
Vanilla RNN	2.27	0%	63%	Best name quality, but memorizes
BLSTM	1.00	100%	50%	Mode collapse (unusable)
RNN Attention	1.00	99%	92%	High diversity, mixed quality

3.5.4 Sample Generated Names

Vanilla RNN (Best Quality, 0% Novel):

ananth, priya, sarabjit, anita, subhadra, gargi, tanushree,
prasad, nanditha, gopalan, palak, hrusikesh, bhavana, vasundha,
krishnan, bharath, vimala, anika, karthik, aditi, sanjukta,
suresh, ganesh, karan, rajesh, pradeep, hemant, pramod, dipali,
kavya, geetha, manorajan, kiran, krishna, hari, kaushik,
saraswathi, charanjit, radha, priyanka, amrita, gauri, vani

Analysis: - All names are real Indian names from training set - High linguistic quality and phonetic correctness - 100% of generated names are recognizable and usable - Issue: Zero creativity, pure memorization - Cause: Likely overfitting due to small dataset and model capacity

BLSTM (Mode Collapse, 100% Novel):

kekekekekekekek, ununununununu, kikikikikikikik,
jojojojojojojoj, ururururururu, gigigigigigigig,
sisisisisisisis, vavavavavavav, dedededededed,
tititititititit, nonononononon, jajajajajajajaj,
kukukukukukuk, didididididid, uuuuuuuuuuuuu,
gagagagagagagag, pipipipipipip, shshshshshshshs,
lililililililil, lelelelelelele, mimimimimimi

Analysis: - Severe mode collapse to repetitive patterns - All names consist of repeated syllables (2 characters) - 0% linguistically valid names - Complete failure of generative capability - Issue: Model learned to repeat single pattern - Possible causes: - Bidirectional processing confuses generation - Training instability - Learning rate too high - Initialization problems

RNN with Attention (High Diversity, Mixed Quality):

Good names:

- zoya (valid Indian name)
- gowan (plausible)
- zabam (phonetically reasonable)
- zopan (could be real)
- fuban (plausible)

Poor names:

- choc (English word, not Indian name)
- bhacububbbbbbb (nonsensical)
- chacbhacbhuhu (random syllables)
- chagchac (repetitive, low quality)
- bhagyayyay (partially valid but flawed)
- chucuc (repetitive)
- chacchucdu (random combination)
- cubap (unclear)
- fubab (unclear)

Analysis: - Mix of plausible and nonsensical names - Some generated names could pass as real (zoya, gowan, zabam) - Many contain nonsensical character combinations - High diversity (92% unique) but inconsistent quality - Shows creativity but lacks linguistic constraints - Issue: No quality control, overly random

3.5.5 Character Distribution Analysis

Training Data: - Most common: a (22%), i (15%), r (12%), n (11%), a (9%) - Rare: q (0.2%), x (0.3%), z (0.4%) - Realistic distribution reflecting Indian name patterns

Vanilla RNN Generated: - Nearly identical to training data - a (22%), i (15%), r (12%), n (11%) - Expected since it memorizes training names - Confirms overfitting diagnosis

BLSTM Generated: - Heavily skewed distribution - Top characters: k (18%), i (12%), u (10%), e (9%) - Missing many common characters from training - Reflects mode collapse to specific patterns

RNN Attention Generated: - More uniform distribution - a (10%), c (9%), h (8%), b (7%), u (7%) - Overuses some consonant clusters (ch-, bh-) - Shows attempt to generate diverse patterns - Less representative of real Indian names

3.5.6 Length Distribution Analysis

Training Data: - Mean: 6.8 characters - Std: 1.9 characters - Min: 3, Max: 15 - Median: 7 - Most names: 5-8 characters

Vanilla RNN Generated: - Mean: 6.7 characters - Std: 1.4 characters - Min: 3, Max: 10 - Nearly matches training distribution - Again confirms memorization

BLSTM Generated: - Mean: 15.0 characters (maximum allowed) - All names hit maximum length limit - No natural stopping - Another sign of mode collapse - Model doesn't learn when to stop

RNN Attention Generated: - Mean: 7.2 characters - Std: 2.8 characters - More variable than training - Some very short (3-4 char) and long (12-15 char) names - Shows more exploration of length space

3.5.7 Model Comparison

Quantitative Summary:

Metric	Vanilla RNN	BLSTM	RNN Attention	Best
Perplexity	2.27	1.00	1.00	BLSTM/Attn
Novelty	0%	100%	99%	BLSTM/Attn
Diversity	63%	50%	92%	Attention
Name Quality	High	Zero	Medium	Vanilla
Usability	Low	Zero	Low-Med	Vanilla
Training Time	10s	12s	15s	Vanilla

Qualitative Summary:

Aspect	Vanilla RNN	BLSTM	RNN Attention
Linguistic Validity	Excellent	None	Mixed
Phonetic Correctness	Perfect	None	Partial
Creativity	None	None	High
Diversity	Medium	Low	High
Consistency	High	N/A	Low
Production Ready	No (memorizes)	No (broken)	No (quality)

Trade-offs:

1. Quality vs Novelty:

- Vanilla RNN: High quality, zero novelty
- Attention: High novelty, mixed quality
- Ideal: Balance both (not achieved)

2. Diversity vs Consistency:

- High diversity (Attention): Inconsistent quality
- Low diversity (Vanilla): Consistent but repetitive
- Need: Diverse yet consistently high quality

3. Perplexity vs Generation:

- Low perplexity (BLSTM): Doesn't guarantee good generation
- Higher perplexity (Vanilla): Can indicate healthier learning
- Lesson: Perplexity alone is insufficient metric

3.5.8 Failure Analysis

Vanilla RNN: Overfitting

Root causes: - Small dataset (5000 names) - High model capacity (256 hidden units, 2 layers) - Insufficient regularization (dropout=0.2 may be too low) - Long training (30 epochs)

Solutions: - Increase dropout to 0.4-0.5 - Add weight decay (L2 regularization) - Use smaller model (128 hidden units) - Earlier stopping - Data augmentation (character-level noise) - Temperature > 1.0 during generation

BLSTM: Mode Collapse

Root causes: - Bidirectional processing incompatible with generation - Training on full sequences but generating auto-regressively - Mismatch between training and inference - Possible gradient issues with bidirectional flow

Solutions: - Use unidirectional LSTM for generation tasks - Train with scheduled sampling (gradually use model's own predictions) - Different initialization strategy - Lower learning rate - Add layer normalization - Increase model capacity

RNN Attention: Quality Inconsistency

Root causes: - High sampling temperature creates randomness - No linguistic constraints during generation - Attention may focus on wrong parts - Small dataset limits learned patterns

Solutions: - Implement beam search (explore multiple hypotheses) - Add character n-gram penalties (avoid nonsensical combinations) - Use language model scoring - Conditional generation (e.g., specify name length) - Ensemble multiple models - Train on larger dataset

3.6 Conclusions

3.6.1 Key Findings

1. Architecture Performance:

- Vanilla RNN produces highest quality names but memorizes training data
- BLSTM suffers from severe mode collapse, unusable for generation
- RNN with Attention shows high diversity but inconsistent quality
- No single architecture solves all requirements

2. Metrics vs Quality:

- Perplexity doesn't correlate with generation quality
- Need multiple metrics: novelty, diversity, AND human evaluation
- Perfect test metrics (perplexity=1.0) can indicate problems
- Generation quality requires manual inspection

3. Overfitting vs Creativity:

- Small dataset leads to overfitting (Vanilla RNN)
- Perfect memorization prevents creative generation
- Need balance between learning patterns and generalizing
- Regularization crucial for small datasets

4. Bidirectional Models:

- Bidirectional processing doesn't suit auto-regressive generation
- Training-inference mismatch causes mode collapse
- Unidirectional models more appropriate for generation
- BLSTM better suited for classification/tagging, not generation

3.6.2 Practical Recommendations

For Best Current Results: Use Vanilla RNN with modifications: - Increase dropout to 0.4 - Add temperature sampling ($T=1.2-1.5$) - Post-process: Filter out training names - Accept some quality for novelty trade-off

For Production System: 1. Fix fundamental issues first: - Solve BLSTM mode collapse - Improve Attention quality consistency - Expand dataset to 10k+ names

2. Implement advanced techniques:
 - Beam search decoding
 - Language model reranking
 - Ensemble of models
 - Conditional generation (gender, region, style)
3. Add quality controls:
 - Phonetic validity checker
 - Cultural appropriateness filter
 - Length constraints
 - Character pattern validation

3.6.3 Limitations

1. **Dataset:**
 - Small size (5000 names) limits generalization
 - No metadata (gender, region, language) for conditional generation
 - May not represent full diversity of Indian names
2. **Evaluation:**
 - Automatic metrics don't capture human perception
 - No user study or preference testing
 - Linguistic validity hard to quantify
 - Cultural appropriateness not measured
3. **Models:**
 - Simple architectures may lack capacity for complex patterns
 - No pre-training or transfer learning
 - Single-task learning (only name generation)
4. **Generation:**
 - No control over style or attributes
 - Binary accept/reject, no fine-grained quality scoring
 - Temperature tuning is manual and heuristic

3.6.4 Future Work

Short-term Improvements:

1. **Fix BLSTM:**
 - Use unidirectional LSTM
 - Different training procedure
 - Scheduled sampling
 - Layer normalization
2. **Improve Vanilla RNN:**
 - Stronger regularization
 - Data augmentation
 - Ensemble methods
 - Better sampling strategies
3. **Enhance Attention:**
 - Beam search
 - N-gram penalties

- Quality scoring
- Reranking

Long-term Enhancements:

1. **Advanced Architectures:**
 - Transformer-based models
 - VAE for name generation
 - GAN for adversarial training
 - Pre-trained language models (fine-tune GPT, BERT)
 2. **Conditional Generation:**
 - Gender-specific names
 - Regional variations
 - Language family (Hindi, Tamil, etc.)
 - Style control (traditional, modern, unique)
 3. **Quality Assurance:**
 - Discriminator network to judge quality
 - Reinforcement learning from human feedback
 - Phonetic similarity constraints
 - Cultural appropriateness classifier
 4. **Dataset Expansion:**
 - Collect 50k+ diverse Indian names
 - Include metadata (gender, region, language)
 - Add name meanings and etymologies
 - Multilingual support
 5. **Evaluation:**
 - Human evaluation studies
 - A/B testing with users
 - Downstream task evaluation
 - Cultural sensitivity assessment
-

Overall Conclusions

Key Achievements

Problem 1: Word2Vec

- Successfully implemented CBOW and Skip-gram from scratch
- Trained 7 model configurations with different hyperparameters
- Comprehensive evaluation using similarity, analogy, and visualization
- Results comparable to standard Gensim implementation
- Deep understanding of word embedding training demonstrated

Problem 2: Character-Level RNN

- Implemented three RNN architectures from scratch
- Trained all models successfully
- Comprehensive evaluation with multiple metrics
- Generated sample names and analyzed patterns
- Identified failure modes and proposed solutions

Technical Skills Demonstrated

1. **Deep Learning Implementation:**

- PyTorch neural network programming
 - Custom model architectures
 - Training loop implementation
 - Gradient computation and optimization
2. **Natural Language Processing:**
 - Text preprocessing and tokenization
 - Word and character embeddings
 - Sequence modeling
 - Language generation
 3. **Experimental Methodology:**
 - Hyperparameter tuning
 - Multiple evaluation metrics
 - Ablation studies
 - Model comparison
 4. **Analysis and Interpretation:**
 - Quantitative results analysis
 - Qualitative output inspection
 - Failure mode identification
 - Practical recommendations

Lessons Learned

1. **Implementation Matters:**
 - Details like initialization, learning rate, and regularization are crucial
 - Debugging requires understanding of underlying theory
 - Simple architectures can be effective with proper tuning
2. **Evaluation is Multifaceted:**
 - Single metrics (perplexity, loss) insufficient
 - Need multiple quantitative and qualitative measures
 - Manual inspection essential for generation tasks
 - Perfect metrics don't guarantee good results
3. **Data is Critical:**
 - Small datasets lead to overfitting
 - Quality and diversity matter as much as quantity
 - Preprocessing choices significantly impact results
 - More data often more valuable than architectural complexity
4. **Trade-offs are Inevitable:**
 - Quality vs. novelty (Problem 2)
 - Speed vs. accuracy
 - Simplicity vs. expressiveness
 - Must choose based on application requirements

Future Directions

Near-term:

- Expand datasets for both problems
- Fix identified issues (BLSTM mode collapse, vanilla RNN overfitting)
- Implement advanced sampling strategies
- Add more evaluation metrics

Long-term:

- Explore modern architectures (Transformers, GPT-style models)
- Transfer learning and pre-training

- Multi-task and multi-lingual extensions
- Production deployment considerations

Final Remarks

This assignment provided hands-on experience with fundamental NLU techniques: - Word embeddings capture semantic relationships - Character-level RNNs can model orthographic patterns - Implementation from scratch deepens understanding - Evaluation requires multiple perspectives

Both problems demonstrate that while neural language models are powerful, they require careful: - Architecture design - Hyperparameter tuning - Training procedures - Evaluation methodology

The insights gained form a solid foundation for advanced NLU work and modern large language models.

References

Problem 1: Word2Vec

1. Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781.
2. Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. In Advances in neural information processing systems (pp. 3111-3119).
3. Goldberg, Y., & Levy, O. (2014). word2vec Explained: deriving Mikolov et al.'s negative-sampling word-embedding method. arXiv preprint arXiv:1402.3722.
4. Rong, X. (2014). word2vec parameter learning explained. arXiv preprint arXiv:1411.2738.
5. Levy, O., & Goldberg, Y. (2014). Neural word embedding as implicit matrix factorization. In Advances in neural information processing systems (pp. 2177-2185).

Problem 2: Character-Level RNN

6. Graves, A. (2013). Generating sequences with recurrent neural networks. arXiv preprint arXiv:1308.0850.
7. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural computation, 9(8), 1735-1780.
8. Schuster, M., & Paliwal, K. K. (1997). Bidirectional recurrent neural networks. IEEE transactions on Signal Processing, 45(11), 2673-2681.
9. Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural machine translation by jointly learning to align and translate. arXiv preprint arXiv:1409.0473.
10. Karpathy, A. (2015). The Unreasonable Effectiveness of Recurrent Neural Networks. Blog post.
11. Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. In Advances in neural information processing systems (pp. 3104-3112).

General Deep Learning

12. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT press.
13. Chollet, F. (2017). Deep learning with Python. Manning Publications.

14. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In Advances in neural information processing systems (pp. 8026-8037).

End of Report

Appendices

Appendix A: Code Structure

Complete source code is organized as follows:

```
NLU/
  README.md           # Main documentation
  FINAL_REPORT.pdf    # This report
  requirements.txt     # Python dependencies
  problem1_word2vec/
    README.md
    config.yaml
    main.py
    word2vec/         # Core implementation
    analysis/         # Evaluation scripts
    scripts/          # Utilities
    data/             # Dataset
    outputs/          # Results
  problem2_name_generation/
    README.md
    config.yaml
    main.py
    models/           # RNN implementations
    training/         # Training utilities
    evaluation/       # Evaluation scripts
    preprocessing/    # Data preprocessing
    data/             # Dataset
    outputs/          # Results
```

Appendix B: Hyperparameters Summary

Problem 1:

```
embedding_dim: 100
window_size: 5
min_count: 3
num_epochs: 50
batch_size: 512
learning_rate: 0.025
negative_samples: 10
```

Problem 2:

```
embedding_dim: 128
hidden_dim: 256
num_layers: 2
dropout: 0.2
```

```
num_epochs: 30
batch_size: 64
learning_rate: 0.001
early_stopping_patience: 7
temperature: 0.8
top_p: 0.9
```

Appendix C: Execution Instructions

Setup:

```
pip install -r requirements.txt
```

Problem 1:

```
cd problem1_word2vec
python main.py --task all
```

Problem 2:

```
cd problem2_name_generation
python main.py --task all
```

Appendix D: Output Files

All results, models, and visualizations are saved in respective **outputs/** directories with timestamps and configuration details for reproducibility.

Submitted by:

Dhruva Kumar Kaushal
B22AI017
Natural Language Understanding